

```

1 from sklearn.model_selection import train_test_split
2 import numpy as np
3
4 X=np.random.randn(10240,8)
5 Y=np.random.randn(10240)
6
7 # æ*°æ@éç,,â¼,,ç†
8 ones=np.ones(shape=(X.shape[0],1))
9 X=np.hstack([X,ones])
10 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, shuffle=True)
11
12 # batchâ±½æ*°
13 def get_batch(batch_size,X,Y):
14     batchnum=X.shape[0]//batch_size
15     X_new=X.reshape((batchnum,batch_size,X.shape[1]))
16     Y_new=Y.reshape(batchnum,batch_size,)
17     for i in range(batchnum):
18         yield X_new[i,:,:],Y_new[i,:]
19
20 # æŸâ¼±â±½æ*°
21 def mse(X,Y,W):
22     return 0.5*np.mean(np.square(X@W-Y))
23
24 def diff_mse(X,Y,W):
25     return X.T@(X@W-Y)/X.shape[0]
26
27 # SGD
28 lr=0.001
29 batch_size=64
30 num_epochs=100
31 evaluate_time=10
32 def train(num_epochs,batch_size,W,X_train, X_test, Y_train, Y_test):
33     loss_train=[]
34     loss_evaluate=[]
35     for epoch in range(num_epochs):
36         loss_train_epoch=0
37         for x_batch,y_batch in get_batch(batch_size,X_train,Y_train):
38             loss_batch=mse(x_batch,y_batch,W)
39             loss_train_epoch += loss_batch*x_batch.shape[0]/X_train.shape[0]
40             grad=diff_mse(x_batch,y_batch,W)
41             W=W-lr*grad
42
43         loss_train.append(loss_train_epoch)
44         if 0==epoch % evaluate_time:
45             loss_evaluate_epoch=mse(X_test,Y_test,W)
46             loss_evaluate.append(loss_evaluate_epoch)
47             print(f'Epoch: {epoch}, train loss: {loss_train_epoch}, val loss: {loss_evaluate_epoch}')
48
49
50 if __name__=="__main__":
51     W=np.random.randn(X.shape[1],)
52     train(num_epochs,batch_size,W,X_train, X_test, Y_train, Y_test)
53

```